Normalisasi Data (BCNF, 3NF, 4NF)

Oleh:
Ir. Hira Laksmiwati Soemitro, M.Sc
Hartanto Tantriawan, S.Kom., M.Kom
Arief Ichwani, S.Kom., M.Cs

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026

Capaian pembelajaran

- Mendeskripsikan definisi dari **3NF, dan BCNF.**
- Mengidentifikasi apakah suatu relasi memenuhi kriteria **1NF, 2NF, 3NF, atau BCNF.**
- Melakukan normalisasi relasi **1NF** menjadi sekumpulan relasi **3NF** (atau **BCNF**) dan melakukan denormalisasi pada skema relasional.
- Menjelaskan dampak normalisasi terhadap efisiensi operasi basis data, khususnya pada optimasi kueri (***query optimization***).

Boyce-Codd Normal Form (BCNF)

Sebuah skema relasi ***R*** dikatakan berada dalam bentuk ***BCNF*** terhadap sekumpulan ketergantungan fungsional ***F***, jika untuk semua ketergantungan fungsional dalam F^+ dengan bentuk:

- $\alpha \rightarrow \beta$

di mana $\alpha \subseteq R$ dan $\beta \subseteq R$, setidaknya salah satu dari kondisi berikut terpenuhi:

- $\alpha \rightarrow \beta$ adalah trivial (i.e., $\beta \subseteq \alpha$)
- α adalah sebuah superkey untuk ***R***

Contoh skema yang *tidak* berada dalam BCNF:

instr_dept (ID, name, salary, dept_name, building, budget)

Karena *dept_name→ building, budget*

berlaku pada relasi ***instr_dept***, namun ***dept_name*** **bukan** merupakan sebuah ***superkey***.

Dekomposisi Skema ke dalam BCNF

Asumsikan terdapat sebuah skema ***R*** dan sebuah ketergantungan non-trivial **$\alpha \rightarrow \beta$** yang menyebabkan pelanggaran aturan BCNF.

Kita melakukan **dekomposisi** (pemecahan) terhadap ***R*** menjadi:

- **$(\alpha \cup \beta)$**
- **$(R - (\beta - \alpha))$**

- **Dalam contoh kita sebelumnya:**
  - **α = *dept_name***
  - **β = *building, budget***

  Maka, skema ***inst_dept*** digantikan oleh:
  - **$(\alpha \cup \beta)$ = (*dept_name, building, budget*)**
  - **$(R - (\beta - \alpha))$ = (*ID, name, salary, dept_name*)**

BCNF dan Pemeliharaan Ketergantungan (*Dependency Preservation*)

- **Batasan** (***Constraints***), termasuk ketergantungan fungsional, memerlukan biaya komputasi yang tinggi untuk diperiksa secara praktis, kecuali jika batasan tersebut hanya berkaitan dengan satu relasi saja.
- Jika pemeriksaan ketergantungan pada setiap relasi individual hasil dekomposisi sudah cukup untuk memastikan bahwa **seluruh** ketergantungan fungsional tetap terpenuhi, maka dekomposisi tersebut dikatakan bersifat **memelihara ketergantungan** (***dependency preserving***).
- Karena pencapaian kondisi BCNF sekaligus pemeliharaan ketergantungan tidak selalu memungkinkan untuk dilakukan secara bersamaan, kita mempertimbangkan bentuk normal yang lebih longgar, yang dikenal sebagai **Bentuk Normal Ketiga (3NF)**.

Bentuk Normal Ketiga (*Third Normal Form* - 3NF)

- Sebuah skema relasi ***R*** dikatakan berada dalam **Bentuk Normal Ketiga (3NF)** jika untuk semua:

$$\alpha \rightarrow \beta \text{ in } F^+$$

  setidaknya salah satu dari kondisi berikut terpenuhi:
  - $\alpha \rightarrow \beta$ bersifat trivial (yaitu, $\beta \in \alpha$)
  - α adalah sebuah ***superkey*** untuk ***R***.
  - Setiap atribut ***A*** dalam $\beta - \alpha$ termuat di dalam sebuah ***candidate key*** untuk ***R***.
    *(**CATATAN**: setiap atribut boleh berada dalam candidate key yang berbeda-beda).*

- **Poin-poin Penting:**
  - Jika sebuah relasi berada dalam **BCNF**, maka ia sudah pasti berada dalam **3NF** (karena dalam **BCNF**, salah satu dari dua kondisi pertama di atas harus terpenuhi).
  - Kondisi ketiga merupakan bentuk **pelonggaran minimal** dari BCNF untuk menjamin **pemeliharaan ketergantungan** (***dependency preservation***) (penjelasan mengenai alasan hal ini akan dibahas kemudian).

Pengujian untuk BCNF

- Untuk memeriksa apakah sebuah ketergantungan non-trivial **$\alpha \rightarrow \beta$** menyebabkan pelanggaran aturan BCNF:
  - Hitung **α^+** (penutupan atribut atau ***attribute closure*** dari **α**), dan
  - Verifikasi apakah hasil penutupan tersebut mencakup seluruh atribut dalam ***R***. Jika ya, maka **α** adalah sebuah ***superkey*** dari ***R***.
- **Pengujian yang Disederhanakan (*Simplified test*):** Untuk memeriksa apakah sebuah skema relasi ***R*** berada dalam BCNF, cukup dengan memeriksa ketergantungan dalam himpunan ***F*** yang diberikan saja untuk melihat adanya pelanggaran BCNF, daripada harus memeriksa seluruh ketergantungan dalam F^+.
  - Jika tidak ada ketergantungan dalam F yang menyebabkan pelanggaran BCNF, maka tidak akan ada ketergantungan dalam F^+ yang menyebabkan pelanggaran BCNF pula.
- **Namun, pengujian yang disederhanakan hanya menggunakan *F* menjadi tidak akurat saat menguji relasi dalam hasil dekomposisi *R*:**
  - **Pertimbangkan**: $R = (A, B, C, D, E)$, dengan $F = \{ A \rightarrow B, BC \rightarrow D\}$
  - Dekomposisi R menjadi $R_1 = (A,B)$ dan $R_2 = (A,C,D, E)$.
  - Tidak ada ketergantungan dalam F yang hanya mengandung atribut dari (A, C, D, E), sehingga kita mungkin keliru menganggap bahwa R_2 sudah memenuhi BCNF.
  - Faktanya, ketergantungan $AC \rightarrow D$ dalam F^+ menunjukkan bahwa R_2 **tidak** berada dalam BCNF.

Menguji Dekomposisi untuk BCNF

- Untuk memeriksa apakah sebuah relasi R_i dalam hasil dekomposisi R berada dalam bentuk **BCNF**:
  - Lakukan pengujian R_i terhadap BCNF dengan merujuk pada **pembatasan (*restriction*)** F ke R_i (yaitu, semua *Functional Dependencies* (*FD*) dalam F^+ yang hanya mengandung atribut-atribut dari R_i).
  - **Atau**, gunakan himpunan ketergantungan asli F yang berlaku pada R, namun dengan pengujian sebagai berikut:
    - Untuk setiap himpunan atribut $\alpha \subseteq R_i$, periksa apakah α^+ (penutupan atribut dari α) tidak mengandung atribut dari $R_i - \alpha$, **atau** mencakup seluruh atribut dalam R_i.
    - Jika kondisi tersebut dilanggar oleh suatu $\alpha \rightarrow \beta$ dalam F, maka ketergantungan:

$$\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$$

    - dapat ditunjukkan berlaku pada R_i, sehingga R_i melanggar aturan BCNF.
    - Kita menggunakan ketergantungan di atas untuk melakukan dekomposisi lebih lanjut pada R_i.

BCNF Decomposition Algorithm

```
result := {R };
done := false;
compute F +;
while (not done) do
          if (there is a schema Ri in result  that is not in BCNF)
                    then begin
                              let α → β  be a nontrivial functional dependency
that
                       holds on Ri  such that α → Ri is not in F +,
                                        and α ∩ β  = Ø;
                                result := (result − Ri ) ∪ (Ri − β) ∪ (α, β );
                    end
                    else done := true;
```

Note: each R_i is in BCNF, and decomposition is lossless-join.

Contoh Dekomposisi BCNF

- **Diketahui:**
  - Skema relasi $R = (A, B, C)$
  - Himpunan ketergantungan fungsional $F = \{A \rightarrow B, B \rightarrow C\}$
  - Kunci (*Key*) = $\{A\}$
- **Analisis:**
  - R **tidak** berada dalam bentuk BCNF karena terdapat ketergantungan $B \rightarrow C$, namun B **bukan** merupakan sebuah ***superkey***.
- **Dekomposisi:**
  - $R_1 = (B, C)$
  - $R_2 = (A,B)$

Contoh Dekomposisi BCNF

- **Skema Relasi *class*:**
  - ***(course_id, title, dept_name, credits, sec_id, semester, year, building, room_number, capacity, time_slot_id)***
- **Ketergantungan Fungsional (*Functional Dependencies*):**
  - ***course_id → title, dept_name, credits***
  - ***building, room_number → capacity***
  - ***course_id, sec_id, semester, year → building, room_number, time_slot_id***
- **Kunci Kandidat (*Candidate Key*):**
  - ***{course_id, sec_id, semester, year}***
- **Proses Dekomposisi BCNF:**
  - Ketergantungan *course_id → title, dept_name, credits* **berlaku(holds)**, namun *course_id* **bukan** merupakan sebuah ***superkey***.
  - Oleh karena itu, kita mengganti relasi ***class*** dengan:
    - **course** *(course_id, title, dept_name, credits)*
    - **class-1** *(course_id, sec_id, semester, year, building, room_number, capacity, time_slot_id)*

Dekomposisi BCNF (Lanjutan)

- **Mari kita periksa relasi-relasi berikut:**
  - ***course*** *(course_id, title, dept_name, credits)*
  - ***class-1*** *(course_id, sec_id, semester, year, building, room_number, capacity, time_slot_id)*
- **Relasi *course* sudah berada dalam BCNF**
  - Bagaimana kita mengetahui hal ini? (Karena satu-satunya penentu, *course_id*, adalah kunci utama).
- **Ketergantungan** *building, room_number → capacity* **berlaku pada *class-1***
  - Namun, *{building, room_number}* **bukan** merupakan sebuah ***superkey*** untuk relasi ***class-1***.
- **Kita mengganti *class-1* dengan:**
  - ***classroom*** *(building, room_number, capacity)*
  - ***section*** *(course_id, sec_id, semester, year, building, room_number, time_slot_id)*
- **Relasi *classroom* dan *section* kini telah berada dalam BCNF.**

BCNF dan Pemeliharaan Ketergantungan (*Dependency Preservation*)

- Tidak selalu memungkinkan untuk memperoleh dekomposisi BCNF yang bersifat memelihara ketergantungan (***dependency preserving***).
- **Diketahui:**
  - Skema relasi $R = (J, K, L)$
  - Himpunan ketergantungan fungsional $F = \{JK \rightarrow L, L \rightarrow K\}$
  - Dua kunci kandidat (*candidate keys*) = JK dan JL
- **Analisis:**
  - R **tidak** berada dalam bentuk BCNF (karena pada ketergantungan $L \rightarrow K$, atribut L bukan merupakan sebuah ***superkey***).
- **Konsekuensi:**
  - Setiap dekomposisi terhadap R akan gagal memelihara ketergantungan $JK \rightarrow L$.
  - Hal ini berimplikasi bahwa pengujian terhadap ketergantungan $JK \rightarrow L$ akan memerlukan operasi penggabungan (***join***).

Bentuk Normal Ketiga: Motivasi

- Terdapat beberapa situasi di mana:
  - **BCNF tidak bersifat memelihara ketergantungan (*dependency preserving*)**, dan
  - Pemeriksaan pelanggaran **Ketergantungan Fungsional (FD)** yang efisien saat terjadi pembaruan data (***update***) sangatlah penting.
- **Solusi:** Mendefinisikan bentuk normal yang lebih longgar, yang disebut sebagai **Bentuk Normal Ketiga (3NF)**.
  - Mengizinkan adanya redundansi tertentu (beserta konsekuensi masalah yang akan ditimbulkan; contoh akan dibahas kemudian).
  - Namun, ketergantungan fungsional dapat diperiksa pada relasi individual tanpa perlu melakukan operasi penggabungan (***join***).
  - Selalu terdapat dekomposisi ke dalam **3NF** yang bersifat ***lossless-join*** (penggabungan tanpa kehilangan data) dan ***dependency-preserving*** (memelihara ketergantungan).

Contoh Bentuk Normal Ketiga (3NF)

- **Relasi *dept_advisor*:**
  - ***dept_advisor*** *(s_ID, i_ID, dept_name)*
  - *F = {s_ID, dept_name → i_ID, i_ID → dept_name}*
- **Dua kunci kandidat (*candidate keys*):**
  - *s_ID, dept_name,* dan *i_ID, s_ID*
- **Relasi R berada dalam 3NF:**
  - **Untuk ketergantungan *s_ID, dept_name → i_ID* :**
    - *s_ID, dept_name* is a **superkey**.
  - **Untuk ketergantungan *i_ID → dept_name* :**
    - Atribut *dept_name* termuat di dalam sebuah ***candidate key*** (yaitu *{s_ID, dept_name}*).

Redundansi pada 3NF

- **3NF** tidak cukup memadai untuk menangani kasus relasi yang:
  - Memiliki dua atau lebih kunci kandidat (***candidate keys***), sedemikian rupa sehingga:
    - Kunci-kunci kandidat tersebut bersifat komposit (terdiri dari lebih dari satu atribut), dan
    - Kunci-kunci tersebut saling tumpang tindih (***overlapping***), yaitu memiliki setidaknya satu atribut yang sama.
- Terdapat beberapa redundansi dalam skema seperti ini.
- **Contoh masalah akibat redundansi pada 3NF:**
  - $R = (J, K, L)$
  - $F = \{JK \rightarrow L, L \rightarrow K\}$
- **Repetisi informasi** (misalnya, hubungan antara l_1 dan k_1):
  - Contoh nyata: pasangan ***(i_ID, dept_name)*** yang muncul berulang kali.
- **Kebutuhan untuk menggunakan nilai *null*** (misalnya, untuk merepresentasikan hubungan l_2 dan k_2 di mana tidak ada nilai yang sesuai untuk J):
  - Contoh nyata: pasangan ***(i_ID, dept_name)*** jika tidak ada relasi terpisah yang memetakan instruktur ke departemen.

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

Pengujian untuk 3NF

- **Optimasi:** Cukup dengan memeriksa **Ketergantungan Fungsional** **(FD)** yang ada dalam himpunan *F*, tidak perlu memeriksa seluruh FD dalam F^+.
- Gunakan ***attribute closure*** (penutupan atribut) untuk memeriksa setiap ketergantungan **$\alpha \rightarrow \beta$**, guna memastikan apakah **α** merupakan sebuah ***superkey***.
- Jika **α** **bukan** merupakan sebuah ***superkey***, kita harus memverifikasi apakah setiap atribut dalam **β** termuat di dalam sebuah ***candidate key*** (kunci kandidat) dari ***R***.
  - Pengujian ini cenderung lebih mahal (secara komputasi), karena melibatkan proses pencarian kunci-kunci kandidat.
  - Pengujian untuk 3NF telah terbukti termasuk dalam kategori permasalahan **NP-hard**.
  - Menariknya, dekomposisi ke dalam bentuk normal ketiga (yang akan dijelaskan segera) dapat dilakukan dalam **waktu polinomial**.

3NF Decomposition Algorithm

```
Let F_c be a canonical cover for F;
i := 0;
for each  functional dependency α → β in F_c do
    if none of the schemas R_j, 1 ≤ j  ≤ i contains  α β
                        then begin
                                    i := i  + 1;
                                    R_i  := α β
                                end
if none of the schemas R_j, 1 ≤ j  ≤ i contains a candidate key for R
    then begin
                            i := i  + 1;
                            R_i := any candidate key for R;
                        end
/* Optionally, remove redundant relations */
repeat
    if any schema R_j is contained in another schema R_k
            then /* delete R_j */
             R_j = R;
             i=i-1;
return (R_1, R_2, ..., R_i)
```

Algoritma Dekomposisi 3NF (Lanjutan)

- **Algoritma sebelumnya menjamin bahwa:**
  - Setiap skema relasi R_i berada dalam bentuk **3NF**.
  - Dekomposisi tersebut bersifat **memelihara ketergantungan** (***dependency preserving***) dan ***lossless-join*** (penggabungan tanpa kehilangan informasi).

Dekomposisi 3NF: Sebuah Contoh

- **Skema Relasi:**
  - *cust_banker_branch* = (*customer_id, employee_id, branch_name, type*)
- **Ketergantungan Fungsional (FD) untuk skema relasi ini adalah:**
  - ***customer_id, employee_id → branch_name, type***
  - ***employee_id → branch_name***
  - ***customer_id, branch_name → employee_id***
- **Pertama, kita menghitung *canonical cover* (F_C):**
  - *branch_name* bersifat ekstraneus (***extraneous***) pada sisi kanan (r.h.s) dari ketergantungan pertama.
  - Tidak ada atribut lain yang bersifat ekstraneus, sehingga kita memperoleh F_C sebagai berikut:
    - ***customer_id, employee_id → type***
    - ***employee_id → branch_name***
    - ***customer_id, branch_name → employee_id***

Contoh Dekomposisi 3NF (Lanjutan)

- **Iterasi *loop* menghasilkan skema 3NF sebagai berikut:**
  - ***(customer_id, employee_id, type)***
  - ***(employee_id, branch_name)***
  - ***(customer_id, branch_name, employee_id)***
  - **Observasi:** Perhatikan bahwa (***customer_id, employee_id, type***) sudah memuat kunci kandidat dari skema asli, sehingga tidak perlu menambahkan skema relasi lebih lanjut.
- **Pada akhir iterasi *loop*, lakukan deteksi dan penghapusan skema yang merupakan bagian (*subset*) dari skema lainnya:**
  - Contoh: (***employee_id, branch_name***) dihapus karena merupakan *subset* dari skema lain.
  - Hasil akhir tidak akan bergantung pada urutan ketergantungan fungsional (FD) yang dipertimbangkan.
- **Skema 3NF hasil penyederhanaan adalah:**
  - ***(customer_id, employee_id, type)***
  - ***(customer_id, branch_name, employee_id)***

Perbandingan BCNF dan 3NF

- **Selalu memungkinkan untuk mendekomposisi sebuah relasi ke dalam sekumpulan relasi yang berada dalam bentuk 3NF sedemikian rupa sehingga:**
  - Dekomposisi bersifat ***lossless*** (tidak ada data yang hilang).
  - Ketergantungan fungsional tetap terpelihara (***dependencies are preserved***).
- **Selalu memungkinkan untuk mendekomposisi sebuah relasi ke dalam sekumpulan relasi yang berada dalam bentuk BCNF sedemikian rupa sehingga:**
  - Dekomposisi bersifat ***lossless*** (tidak ada data yang hilang).
  - Namun, **mungkin tidak memungkinkan** untuk memelihara ketergantungan fungsional (***preserve dependencies***).

Seberapa Baik BCNF?

- Terdapat skema basis data dalam bentuk **BCNF** yang tampaknya belum ternormalisasi secara memadai.
- **Pertimbangkan sebuah relasi:**
  - ***inst_info*** *(ID, child_name, phone)*
  - Di mana seorang instruktur dapat memiliki lebih dari satu nomor telepon dan dapat memiliki lebih dari satu anak.

ID	*child_name*	*phone*
99999	David	512-555-1234
99999	David	512-555-4321
99999	William	512-555-1234
99999	Willian	512-555-4321

inst_info

Seberapa Baik BCNF? (Lanjutan)

- Tidak terdapat ketergantungan fungsional (*functional dependencies*) yang non-trivial, oleh karena itu relasi tersebut berada dalam bentuk **BCNF**.
- **Anomali Insersi** (***Insertion Anomalies***) — sebagai contoh, jika kita menambahkan nomor telepon 981-992-3443 untuk ID 99999, maka kita perlu menambahkan dua tupel (baris):
  - (99999, David, 981-992-3443)
  - (99999, William, 981-992-3443)

Seberapa Baik BCNF? (Lanjutan)

- Oleh karena itu, lebih baik untuk melakukan dekomposisi terhadap relasi ***inst_info*** menjadi:

inst_child

ID	*child_name*
99999	David
99999	David
99999	William
99999	Willian

inst_phone

ID	*phone*
99999	512-555-1234
99999	512-555-4321
99999	512-555-1234
99999	512-555-4321

- Hal ini menunjukkan perlunya bentuk normal yang lebih tinggi, seperti **Bentuk Normal Keempat (4NF)**.
- **4NF** menggunakan konsep **Ketergantungan Multi-nilai (*Multivalued Dependencies*)**

Tujuan Desain

- **Tujuan utama dari desain basis data relasional adalah:**
  - Mencapai **BCNF**.
  - Bersifat ***Lossless join*** (penggabungan tanpa kehilangan data).
  - Memelihara ketergantungan (***Dependency preservation***).
- **Jika kita tidak dapat mencapai semua tujuan tersebut, kita menerima salah satu kondisi berikut:**
  - Kurangnya pemeliharaan ketergantungan (*lack of dependency preservation*).
  - Adanya redundansi akibat penggunaan **3NF**.
- **Hal yang menarik, SQL tidak menyediakan cara langsung untuk menentukan ketergantungan fungsional selain melalui *superkeys*:**
  - Kita dapat menentukan ketergantungan fungsional (FD) menggunakan ***assertions***, namun sangat mahal untuk diuji (dan saat ini tidak didukung oleh sebagian besar basis data yang umum digunakan!).
- **Bahkan jika kita memiliki dekomposisi yang memelihara ketergantungan, dengan menggunakan SQL kita tetap tidak akan mampu menguji secara efisien suatu ketergantungan fungsional yang sisi kirinya (*left hand side*) bukan merupakan sebuah kunci (*key*).**

Pengujian FD Lintas Relasi

- Jika sebuah dekomposisi tidak bersifat memelihara ketergantungan (*dependency preserving*), kita dapat membuat sebuah ***materialized view*** tambahan untuk setiap ketergantungan **$\alpha \rightarrow \beta$ in F_c** yang tidak terpelihara dalam dekomposisi tersebut.
- ***Materialized view*** tersebut didefinisikan sebagai proyeksi pada **α β** dari hasil penggabungan (*join*) relasi-relasi dalam dekomposisi.
- Banyak sistem basis data terbaru yang mendukung ***materialized views***, di mana sistem basis data akan memperbarui *view* tersebut secara otomatis saat relasi-relasi asalnya diperbarui.
  - Tidak diperlukan upaya pemrograman (*coding*) tambahan bagi pemrogram.
- Ketergantungan fungsional **$\alpha \rightarrow \beta$** dinyatakan dengan mendeklarasikan **α** sebagai ***candidate key*** pada ***materialized view*** tersebut.
- Memeriksa ***candidate key*** jauh lebih murah (efisien) dibandingkan dengan memeriksa ketergantungan **$\alpha \rightarrow \beta$**.
- **NAMUN:**
  - ***Space overhead***: Dibutuhkan ruang penyimpanan tambahan untuk menyimpan ***materialized view***.
  - ***Time overhead***: Perlu menjaga agar ***materialized view*** tetap mutakhir (*up to date*) saat relasi-relasi diperbarui.
  - Sistem basis data mungkin tidak mendukung deklarasi kunci (*key declarations*) pada ***materialized views***.

Proses Desain Basis Data Secara Keseluruhan

- **Kita berasumsi bahwa skema *R* telah diberikan:**
  - ***R*** mungkin dihasilkan saat mengonversi diagram E-R (*Entity-Relationship*) ke dalam sekumpulan tabel.
  - ***R*** mungkin berupa relasi tunggal yang berisi seluruh atribut yang diperlukan (disebut sebagai **relasi universal**).
    - Normalisasi memecah ***R*** menjadi relasi-relasi yang lebih kecil.
  - ***R*** mungkin merupakan hasil dari beberapa desain relasi *ad hoc*, yang kemudian kita uji atau konversi ke dalam bentuk normal.

Model ER dan Normalisasi

- Ketika sebuah diagram E-R dirancang dengan teliti serta mengidentifikasi seluruh entitas dengan benar, tabel-tabel yang dihasilkan dari diagram E-R tersebut seharusnya tidak memerlukan normalisasi lebih lanjut.
- Namun, dalam desain dunia nyata (yang tidak sempurna), dapat muncul ketergantungan fungsional dari atribut non-kunci sebuah entitas ke atribut lainnya dalam entitas yang sama.
  - **Contoh:** Sebuah entitas *employee* (karyawan) memiliki atribut *department_name* (nama departemen) dan *building* (gedung), serta terdapat ketergantungan fungsional:
  - ***department_name→ building***
  - Desain yang baik seharusnya menjadikan *department* sebagai entitas tersendiri.
- Ketergantungan fungsional dari atribut non-kunci pada sebuah *relationship set* (himpunan relasi) mungkin saja terjadi, namun jarang—mengingat sebagian besar relasi bersifat biner.

Denormalisasi untuk Performa

- Ada kalanya kita ingin menggunakan skema yang tidak ternormalisasi demi alasan **performa**.
- **Contoh:** Menampilkan data prasyarat (*prereqs*) bersama dengan ***course_id*** dan ***title*** membutuhkan operasi penggabungan (***join***) antara relasi ***course*** dengan ***prereq***.
- **Alternatif 1: Menggunakan relasi yang didenormalisasi** yang berisi atribut-atribut dari *course* sekaligus *prereq* dengan seluruh atribut di atas.
  - Pencarian (***lookup***) menjadi lebih cepat.
  - Membutuhkan ruang penyimpanan tambahan dan waktu eksekusi ekstra untuk proses pembaruan (***updates***).
  - Beban kerja pemrograman bertambah bagi pemrogram dan adanya kemungkinan kesalahan pada kode tambahan tersebut.
- **Alternatif 2: Menggunakan *materialized view* yang didefinisikan sebagai:**
- **course $\bowtie$ prereq**
  - Keuntungan dan kekurangannya sama dengan di atas, kecuali tidak ada beban kerja pemrograman tambahan bagi pemrogram dan dapat menghindari potensi kesalahan kode.

Masalah Desain Lainnya

- Beberapa aspek dalam desain basis data tidak dapat ditangani hanya dengan normalisasi.
- **Contoh desain basis data yang buruk dan harus dihindari:** Alih-alih menggunakan format **earnings** (***company_id, year, amount***), hindari penggunaan:
  - **earnings_2004, earnings_2005, earnings_2006**, dst., yang semuanya menggunakan skema (***company_id, earnings***).
    - Format di atas memang memenuhi **BCNF**, namun membuat kueri lintas tahun menjadi sulit dan membutuhkan pembuatan tabel baru setiap tahunnya.
  - **company_year (*company_id, earnings_2004, earnings_2005, earnings_2006*)**.
    - Format ini juga memenuhi **BCNF**, tetapi tetap menyulitkan kueri lintas tahun dan membutuhkan penambahan atribut baru setiap tahunnya.
    - Ini merupakan contoh dari ***crosstab***, di mana nilai dari satu atribut dijadikan sebagai nama kolom.
    - Umumnya digunakan dalam ***spreadsheet*** dan alat analisis data.

Bentuk Normal Keempat (4NF)

- **Bentuk Normal Keempat (4NF)** adalah tahapan normalisasi tingkat lanjut yang digunakan untuk menangani masalah yang tidak bisa diselesaikan oleh **BCNF**. Fokus utama 4NF **bukan** lagi pada Ketergantungan Fungsional (*Functional Dependency*), melainkan pada **Ketergantungan Multi-nilai (*Multi-valued Dependency*)**.

1. **Syarat Utama 4NF**
   - Sebuah relasi dikatakan berada dalam bentuk **4NF** jika dan hanya jika:
     - Relasi tersebut sudah memenuhi syarat **BCNF**.
     - Relasi tersebut **tidak memiliki** ketergantungan multi-nilai (***Multi-valued Dependency* - MVD**) yang **non-trivial**.

Bentuk Normal Keempat (4NF)

2. Memahami Multi-valued Dependency (MVD)

- Ketergantungan Multi-nilai terjadi ketika keberadaan satu atau lebih baris dalam tabel menyiratkan keberadaan baris lain dalam tabel yang sama.
- **Notasi: α β □**
  - Artinya, untuk satu nilai **α**, terdapat sekumpulan nilai **β** yang terkait, namun kumpulan nilai **β** tersebut **independen** (tidak berhubungan) dengan atribut lain dalam tabel tersebut.
  - **Analogi:** Bayangkan seorang Dosen yang mengampu banyak **Mata Kuliah** dan memiliki banyak **Hobi**. Antara Mata Kuliah dan Hobi tidak ada hubungan langsung, tetapi keduanya terikat pada Dosen yang sama.

Bentuk Normal Keempat (4NF)

3. Contoh Kasus: Pelanggaran 4NF

- Misalkan kita memiliki tabel **Dosen_Info** sebagai berikut:

ID_Dosen	Mata_Kuliah	Hobi
D001	Basis Data	Membaca
D001	Basis Data	Berenang
D001	Pemrograman	Membaca
D001	Pemrograman	Berenang

Masalah pada tabel di atas adalah:

- **Redundansi Eksponensial:** Jika Dosen D001 menambah 1 hobi baru, kita harus menambah 2 baris baru (satu untuk setiap mata kuliah).
- **Anomali Insersi:** Jika Dosen D001 mulai mengajar mata kuliah baru, kita harus memasangkannya dengan semua hobi yang ada.
- **Kondisi:** Tabel ini sudah memenuhi BCNF karena **tidak** ada ***Functional Dependency*** (Mata_Kuliah tidak menentukan Hobi, dan sebaliknya). Namun, terdapat **MVD**:
  - *ID_Dosen* $\Rightarrow$ *Mata_Kuliah*
  - *ID_Dosen* $\Rightarrow$ *Hobi*

Studi Kasus

Bentuk Unnormalized (UNF)

- Tabel berikut ini **buruk** karena salah satu kolom berikut (**Sertifikasi** dan **Tools**) berisi banyak nilai (**multivalued**).

ID_Karyawan	Nama_Karyawan	Dept_ID	Nama_Dept	Lokasi_Gedung	Proyek_ID	Nama_Proyek	Peran_Proyek	Sertifikasi	Tools_Keahlian
K01	Asta	D10	Riset	Gedung Alfa	P99	AI-Chatbot	Lead	AWS, GCP	Python, Java
K02	Budi	D20	Data	Gedung Beta	P88	Big-Data	Analyst	Azure	SQL, R

Tahap 1NF (Menghilangkan Grup Berulang)

- Atribut harus atomik. Kita memecah baris sehingga setiap sel hanya berisi satu data.

ID_Karyawan	Nama_Karyawan	Dept_ID	Nama_Dept	Lokasi_Gedung	Proyek_ID	Nama_Proyek	Peran_Proyek	Sertifikasi	Tools
K01	Asta	D10	Riset	Gedung Alfa	P99	AI-Chatbot	Lead	AWS	Python
K01	Asta	D10	Riset	Gedung Alfa	P99	AI-Chatbot	Lead	AWS	Java
K01	Asta	D10	Riset	Gedung Alfa	P99	AI-Chatbot	Lead	GCP	Python
K01	Asta	D10	Riset	Gedung Alfa	P99	AI-Chatbot	Lead	GCP	Java
K02	Budi	D20	Data	Gedung Beta	P88	Big-Data	Analyst	Azure	SQL
K02	Budi	D20	Data	Gedung Beta	P88	Big-Data	Analyst	Azure	R

- **Masalah 1NF:** Tabel menjadi sangat panjang dan redundan. *Primary Key* (PK) gabungan di sini sangat gemuk: **(ID_Karyawan, Proyek_ID, Sertifikasi, Tools).**

Tahap 2NF (Menghilangkan Ketergantungan Parsial)

Atribut non-kunci harus bergantung pada **seluruh** PK, bukan hanya Sebagian :

- Nama_Karyawan, Dept_ID, Nama_Dept, dan Lokasi_Gedung hanya butuh ID_Karyawan (Parsial).
- Nama_Proyek hanya butuh Proyek_ID (Parsial).
- Peran_Proyek butuh kombinasi ID_Karyawan dan Proyek_ID.

Hasil Dekomposisi 2NF:

- **Tabel Karyawan_Dept:** (ID_Karyawan, Nama_Karyawan, Dept_ID, Nama_Dept, Lokasi_Gedung)
- **Tabel Proyek:** (Proyek_ID, Nama_Proyek)
- **Tabel Tugas_Proyek:** (ID_Karyawan, Proyek_ID, Peran_Proyek)
- **Tabel Keahlian_Sertifikasi:** (ID_Karyawan, Sertifikasi, Tools)

Hasil 2NF

1. Tabel Karyawan_Dept

ID_Karyawan	Nama_Karyawan	Dept_ID	Nama_Dept	Lokasi_Gedung
K01	Asta	D10	Riset	Gedung Alfa
K02	Budi	D20	Data	Gedung Beta

2. Tabel Proyek

Proyek_ID	Nama_Proyek
P99	AI-Chatbot
P88	Big-Data

3. Tabel Tugas_Proyek

ID_Karyawan	Proyek_ID	Peran_Proyek
K01	P99	Lead
K02	P88	Analyst

4. Tabel Keahlian_Sertifikasi

ID_Karyawan	Sertifikasi	Tools
K01	AWS	Python
K01	AWS	Java
K01	GCP	Python
K01	GCP	Java
K02	Azure	SQL
K02	Azure	R

Tahap 3NF (Menghilangkan Ketergantungan Transitif)

- Pada **Tabel_Karyawan_Dept**, Lokasi_Gedung sebenarnya **bergantung** pada Nama_Dept, dan Nama_Dept **bergantung** pada Dept_ID. Ini adalah **transitif**.
- **Hasil Dekomposisi 3NF:**
  - **Tabel_Karyawan:** (ID_Karyawan, Nama_Karyawan, **Dept_ID**)
  - **Tabel_Departemen:** (Dept_ID, Nama_Dept, Lokasi_Gedung)
  - *(Tabel Proyek dan Tugas_Proyek tetap sama)*

Hasil 3NF

1. Tabel Karyawan

ID_Karyawan	Nama_Karyawan	Dept_ID
K01	Asta	D10
K02	Budi	D20

2. Tabel Departemen

Dept_ID	Nama_Dept	Lokasi_Gedung
D10	Riset	Gedung Alfa
D20	Data	Gedung Beta

3. Tabel Proyek

Proyek_ID	Nama_Proyek
P99	AI-Chatbot
P88	Big-Data

4. Tabel Tugas_Proyek

ID_Karyawan	Proyek_ID	Peran_Proyek
K01	P99	Lead
K02	P88	Analyst

5. Tabel Keahlian_Sertifikasi

ID_Karyawan	Sertifikasi	Tools
K01	AWS	Python
K01	AWS	Java
K01	GCP	Python
K01	GCP	Java
K02	Azure	SQL
K02	Azure	R

Tahap BCNF (Boyce-Codd Normal Form)

- Misalkan dalam **Tabel_Tugas_Proyek**, ada aturan:
  - setiap Peran_Proyek di proyek tertentu hanya boleh dipegang oleh satu ID_Karyawan unik,
  - dan Peran_Proyek menentukan Proyek_ID.
  - Jika Peran_Proyek (bukan **superkey**) menentukan Proyek_ID, maka langgar **BCNF**.

- **Hasil Dekomposisi BCNF:**
  - Kita pastikan setiap penentu (***determinant***) adalah kunci kandidat. Biasanya dilakukan dengan memisahkan hubungan peran tersebut ke tabel referensi tersendiri.

Hasil BCNF

1. Tabel Karyawan

ID_Karyawan	Nama_Karyawan	Dept_ID
K01	Asta	D10
K02	Budi	D20

2. Tabel Departemen

Dept_ID	Nama_Dept	Lokasi_Gedung
D10	Riset	Gedung Alfa
D20	Data	Gedung Beta

3. Tabel Proyek

Proyek_ID	Nama_Proyek
P99	AI-Chatbot
P88	Big-Data

4. Tabel Tugas_Proyek

ID_Karyawan	Proyek_ID
K01	P99
K02	P88

5. Tabel Keahlian_Sertifikasi

ID_Karyawan	Sertifikasi	Tools
K01	AWS	Python
K01	AWS	Java
K01	GCP	Python
K01	GCP	Java
K02	Azure	SQL
K02	Azure	R

6. Tabel Peran_Proyek

ID_Peran_Proyek	Peran_Proyek
PR01	Lead
PR02	Analyst

Tahap 4NF (Menghilangkan Multi-valued Dependency)

- Perhatikan **Tabel_Keahlian_Sertifikasi** (ID_Karyawan, Sertifikasi, Tools). Seorang karyawan memiliki banyak sertifikasi dan banyak keahlian alat. **Sertifikasi tidak ada hubungannya** dengan **Tools**, tapi **keduanya bergantung** pada **Karyawan**. Ini menyebabkan **kombinasi** baris yang **tidak perlu** (lihat baris K01 di 1NF yang meledak jumlahnya).
- **Hasil Akhir (4NF):**
  - **Tabel Sertifikasi_Karyawan:** (ID_Karyawan, Sertifikasi)
  - **Tabel Tools_Karyawan:** (ID_Karyawan, Tools)

Hasil 4NF(1)

1. Tabel Karyawan

ID_Karyawan	Nama_Karyawan	Dept_ID
K01	Asta	D10
K02	Budi	D20

2. Tabel Departemen

Dept_ID	Nama_Dept	Lokasi_Gedung
D10	Riset	Gedung Alfa
D20	Data	Gedung Beta

3. Tabel Proyek

Proyek_ID	Nama_Proyek
P99	AI-Chatbot
P88	Big-Data

4. Tabel Tugas_Proyek

ID_Karyawan	Proyek_ID
K01	P99
K02	P88

5. Tabel Peran_Proyek

ID_Peran_Proyek	Peran_Proyek
PR01	Lead
PR02	Analyst

Hasil 4NF(2)

6. Tabel Sertifikasi_Karyawan

ID_Karyawan	Sertifikasi
K01	AWS
K01	GCP
K02	Azure

7. Tabel Tools_Karyawan

ID_Karyawan	Tools
K01	Python
K01	Java
K02	SQL
K02	R

Struktur Relasi

1. **Tabel Departemen & Karyawan (Relasi 1:N):**
   - Satu **Departemen** memiliki banyak **Karyawan**.
   - Dept_ID di tabel Karyawan menjadi *Foreign Key* (FK) yang merujuk ke tabel Departemen. Ini menghilangkan ketergantungan transitif (3NF).
2. **Tabel Karyawan & Proyek (Relasi M:N melalui Penugasan):**
   - Seorang **Karyawan** bisa mengerjakan banyak **Proyek**, dan satu Proyek bisa dikerjakan banyak Karyawan.
   - Hubungan ini dijembatani oleh tabel **Penugasan** (atau **Tabel_Tugas_Proyek**) yang menyimpan ID_Karyawan, Proyek_ID, dan atribut unik yaitu Peran_Proyek.
3. **Tabel Sertifikasi & Tools (Dekomposisi 4NF):**
   - **Tabel_Sertifikasi** dan **Tabel_Tools** berdiri sendiri-sendiri sebagai tabel yang merujuk ke ID_Karyawan.
   - **Mengapa dipisah?** Karena Sertifikasi dan Tools tidak memiliki hubungan fungsional satu sama lain. Dengan memisahkan keduanya, kita **menghindari redundansi eksponensial** (misalnya: jika punya 5 sertifikasi dan 5 tools, kita tidak perlu menyimpan **25** baris kombinasi, melainkan hanya 10 baris terpisah).

Manfaat Desain Ini:

- **Update**: Jika lokasi gedung departemen berubah, Anda hanya perlu mengubah 1 baris di tabel Departemen.
- **Insert**: Anda bisa menambahkan data proyek baru meskipun belum ada karyawan yang ditugaskan ke sana.
- **Delete**: Jika seorang karyawan berhenti, Anda cukup menghapus datanya di tabel Karyawan, dan *constraint* (aturan) database akan membersihkan penugasannya tanpa merusak data Proyek itu sendiri.

Tugas Latihan studi kasus

- **Studi Kasus: Sistem Manajemen Klinik Kesehatan (Klinik Sehat)**
- Bayangkan sebuah klinik yang mencatat seluruh aktivitas medis, data pasien, dokter, obat yang diresepkan, dan alergi pasien dalam **satu tabel besar** yang sangat buruk desainnya.

ID_Pasien	Nama_Pasien	Tgl_Lahir	ID_Dokter	Nama_Dokter	Spesialisasi	Ruangan	No_Rekam_Medis	Tgl_Periksa	Obat_Diberikan	Alergi_Pasien
P01	Andi	1990-05-10	D77	Dr. Siti	Anak	Mawar	RM-101	2026-01-20	Paracetamol, Amoxicillin	Debu, Udang
P02	Budi	1985-11-12	D88	Dr. Joko	Umum	Melati	RM-102	2026-01-21	Antasida	Telur

- Lakukan normalisasi yang dibutuhkan (1NF, 2NF, BCNF, 3NF, 4NF)
- Tuliskan semua proses nya, (setiap tabel di setiap step normalisasi nya)
- Analisis apakah tabel hasil akhirnya sama seperti tabel berikut ini? Jika Ya Jelaskan mengapa, jika tidak jelaskan juga mengapa tidak sama!
  - **Tabel_Pasien** (Data Induk Pasien)
  - **Tabel_Dokter** (Data Induk Dokter)
  - **Tabel_Spesialisasi** (Mapping Spesialisasi ke Ruangan)
  - **Tabel_Kunjungan** (Mencatat No Rekam Medis dan Tanggal Periksa)
  - **Tabel_Resep** (Obat apa yang diminum pasien)
  - **Tabel_Alergi** (Daftar alergi tiap pasien secara independen)

Kuis 1

Arigatou Gozaimasu